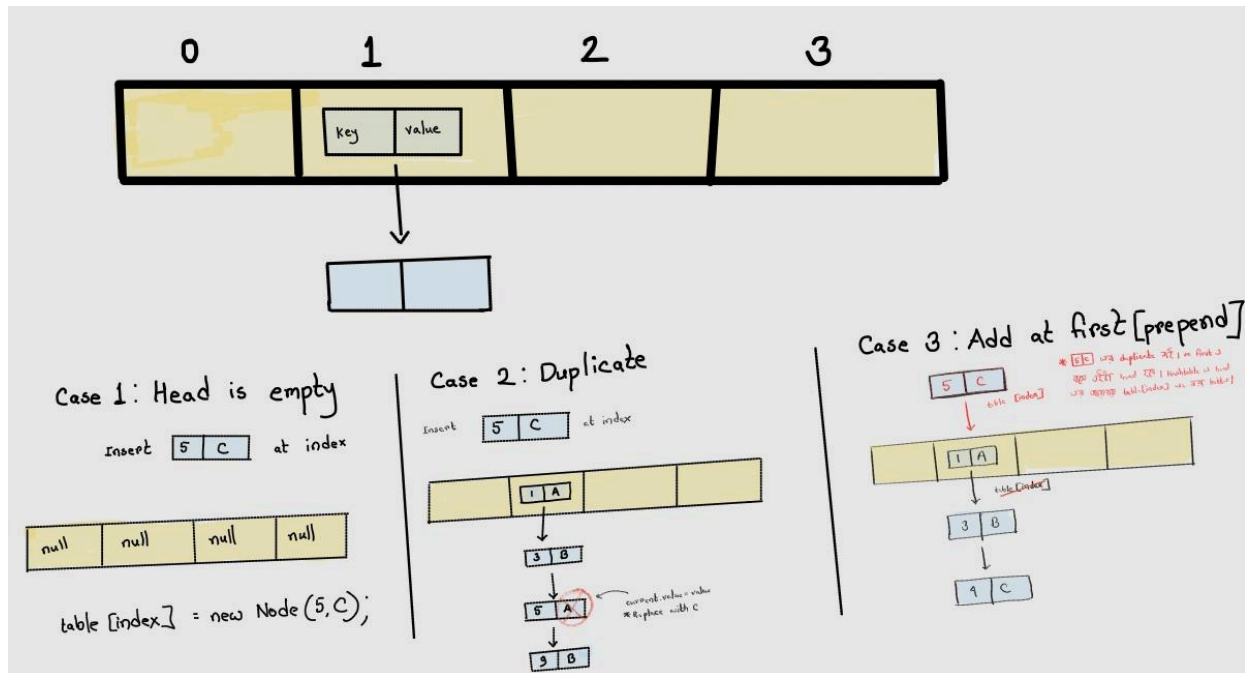


Hashtable Basics

Inserting in a hashtable



// Hash Function

```
private int hash(int key) {
    return key % table.length;
}
```

// Insert function (with chaining)

```
public void insert(int key, int value) {
    int index = hash(key);
    Node head = table[index]; //Linked list create hobe index e
```

// 1

```
if (head == null) {
    table[index] = new Node(key, value);
    return;
}
```

// 2 (If key exists)

```
Node current = head;
```

```

while (current != null) {
    if (current.key == key) {
        current.value = value; // Update existing value
        return;
    }
    current = current.next;
}

// 3
Node newNode = new Node(key, value);
newNode.next = table[index]; // Opor theke add
table[index] = newNode; // Basically turn it into head
}

```

Remove function

```

public void remove(int key) {
    int index = hash(key);
    Node current = table[index];
    Node prev = null;

    while (current != null) {
        if (current.key == key) {
            // If removing head node
            if (prev == null) {
                table[index] = current.next;
            }
            else {
                prev.next = current.next;
            }
            return;
        }
        prev = current;
        current = current.next;
    }
}

```

```
    prev = current;
    current = current.next;
}
}
```

Searching

```
public Integer search(int key) {
    int index = hash(key);
    Node current = table[index];

    while (current != null) {
        if (current.key == key) {
            return current.value;    // Found
        }
        current = current.next;
    }
    return null;    // Not found
}
```

Searching (Array/Pair Node as input)

//Key and value duita array teh stored. Index [0] has key. Index [1] has value

```
public String searchHashtable( Object[] keyValuePair ){
    int indx=hashFunction(keyValuePair[0]);
    PairNode current=ht[indx];

    while(current!=null){
        if(current.key==keyValuePair[0]){
            return "Found" ;
        }
    }
}
```

```
    }  
    current=current.next;  
}  
return "Not found";  
}
```

Print

```
public void printTable() {  
    for (int i = 0; i < table.length; i++) {  
        System.out.print("Index " + i + ": ");  
  
        Node current = table[i];  
        if (current == null) {  
            System.out.println("empty");  
            continue;  
        }  
  
        while (current != null) {  
            System.out.print "[" + current.key + "," + current.value + "] -> ");  
            current = current.next;  
        }  
        System.out.println("null");  
    }  
}
```

Hash Function

```
Integer.parseInt(temp)           //String -> int
Character.getNumericValue(temp)  //Char -> int
int a= key.charAt(i)            //Char -> ASCII
```

hashFunction() → this instance method takes a key-value pair (string, int), calculates the hashed index on key and returns the index. This hash function takes consecutive two letters of the key string, concatenates their ascii values into an integer and sums all the concatenated integers. Then it finds out the modulus of the summation (**think for yourself with which number should we mod the summation**) as the hashed index.

For instance, for a string 'Mortis', the consecutive two letters are Mo, rt, is.

The concatenated integer for

Mo is 77111 (Ascii of M is 77, o is 111);

rt is 114116 (Ascii of r is 114, t is 116);

'is' is 105115 (Ascii of i is 105, s is 115).

The summation is = 77111+114116+105115

Mod the summation with ____ and return the answer as the hashed index.

[fill in the ____ gap with your knowledge]

```
private int hashFunction( String key ){
    int sum=0;
    for(int i=0; i<key.length();i=i+2){
        int b=key.charAt(i+1);
        int a=key.charAt(i);

        String temp= "" + a+b;
        sum+=Integer.parseInt(temp);
    }
    return (int)sum%ht.length;
}
```

Lab task

Lab Tasks [No Need to Submit]

1. Hashtable: Insert using Forward Chaining:

Write the **insert()** function and **hash_function()** of the Hashtable class which uses an array to store a key-value pair, where the key is a string representing a fruit, and the value is an int representing its respective price.

hash_function(key)

Takes a key which is a string, and calculates its length. If length is even, the sum of the ascii values of the even characters is calculated, otherwise, the sum of the ascii values of odd characters is calculated.

Finally, it returns the sum modded by length of the array

If we call `__hash_function("apple")`:

As `len("apple")` is odd, characters in odd indexes (p, l) are taken:

$\text{sum} = \text{ord}(p) + \text{ord}(l) = 112 + 108 = 220$

If the Hashtable object is initialized with length 5, then:

$\text{sum} \% \text{length of Hashtable array} = 220 \% 5 = 0$

So, the function returns 0

insert(key, value)

Creates a node that contains a *tuple(for python) or an array(for java)* which stores the key-value pair in 0,1 index respectively. Finds index by passing key to `hash_function()`. If there is no collision, the node is placed in the index.

If there is a collision, forward chaining is applied. Here, the chain should be arranged in **descending order**, i.e.: if the node being inserted has the highest value (price), it will be the head of the chain. Otherwise, you should iterate the chain and insert it in the appropriate position.

[If the same key is given twice for insertion, update the value of that particular key. No need to create new Node]

```
public class FruitNode {  
  
    Object[] fruit = new Object[2];  
    FruitNode next;  
  
    public FruitNode(String name, Integer price){  
        fruit[0]= name;  
        fruit[1]= price;  
    }  
  
}
```

```

private int hash_function(String key) {
    int len = key.length();
    int sum = 0;

    for (int i = 0; i < len; i++) {
        if (len % 2 == 0 && i % 2 == 0) {
            sum += key.charAt(i);
        }
        else if (len % 2 != 0 && i % 2 != 0) {
            sum += key.charAt(i);
        }
    }

    return sum % table.length;
}

```

```

public void insert(String key, int value) {
    int index = hash_function(key);
    Node head = table[index];

    // Case 1: If key already exists -> update value
    Node current = head;
    while (current != null) {
        if (current.key.equals(key)) {
            current.value = value;
            return;
        }
        current = current.next;
    }
}

```

```

// Case 2
Node newNode = new Node(key, value);

if (head == null || value > head.value) {
    newNode.next = head;
}

```

```
    table[index] = newNode;
    return;
}

// Otherwise find correct descending position
current = head;
while (current.next != null && current.next.value >= value) {
    current = current.next;
}

newNode.next = current.next;
current.next = newNode;
}
```

Lab Assignment

You are given the hash function, $h(\text{key}) = (\text{key} + 3) \% 6$ for a hash-table of length 6. In this hashing, forward chaining is used for resolving conflict and a 6-length array of singly linked lists is used as the hash-table. In the singly linked list, each node has a next pointer, an Integer key and a string value, for example: (4 (key) , "Rafi" (value)). The hash-table stores this key-value pair.

Implement a function **remove(hashTable, key)** that takes a key and a hash-table as parameters. The function removes the key-value pair from the aforementioned hashtable if such a key-value pair (whose key matches the key passed as argument) exists in the hashtable.

Consider, Node class and hashTable are given to you. You just need to complete the remove(hashTable, key) function.

class Node:

```
def __init__(self, key, value, next=None):
    self.key, self.value, self.next = key, value, next
```

Sample Input and Output:

Some Key-value pairs:

(34, "Abid") , (4, "Rafi"), (6, "Karim"), (3, "Chitra"), (22, "Nilu")

HashTable is given in the following:

0: (3, "Chitra")

1: (22, "Nilu") → (4, "Rafi") → (34, "Abid")

2: None

3: (6, "Karim")

4: None

5: None

remove(hashTable, key=4) returns the changed hashTable where (4, "Rafi") is removed.

```
private int hashFunction( Integer key ){
    int indx=(key+3)%ht.length;
    return indx;
}

public void remove( Integer key ){
    int indx=hashFunction(key);
    PairNode head=ht[indx];

    PairNode current=head;
    PairNode prev=null;

    while(current!=null){
        //Judi first ei key pai
        if(prev==null && current.key.equals(key)){
            ht[indx]=current.next;
            return;
        }
        else if(prev!=null && current.key.equals(key)){
```

```
        prev.next=current.next ;  
        return;  
    }  
    prev=current;  
    current=current.next;  
}  
  
return;  
}  
  
}
```

RFTS Practice

You are going to design a Hashtable with forward chaining. So each index of the hash table contains a non dummy headed singly linear linked list to resolve collision. **Your task is to implement the following function with the following requirements.**

- **insert_HashTable:** This function takes a key, a value, and the hash table as parameters and inserts the pair in its appropriate position. However, the insertion is not straightforward. It follows the following conditions.
 - If the key already exists, only update the value.
 - Otherwise
 - If the value is even: insert it at the beginning of the linked list.
 - If the value is odd: insert it at the end of the linked list.

The Node class for the hash table is already implemented with key (String), val (int), and next (Node) variable / attribute.

The hash function is already implemented as named **hash_Function**. You need to use it during insertion.. You need to use it during insertion.

<pre># Python notation def hash_Function(key): # use this function # already written def insert_HashTable(key, value, ht): # Write your code here</pre>	<pre>// Java notation int hash_Function(String key){ // use this function // already written } void insert_HashTable(String key, int value, Node[] ht){ // Write your code here }</pre>
---	--

```
void insert_HashTable(String key, int value, Node[] ht) {
    int index = hash_Function(key);
    Node head = ht[index];
```

```
    if (head == null) {
        ht[index] = new Node(key, value);
        return;
    }
```

```
    Node current = head;
    while (current != null) {
        if (current.key.equals(key)) {
            current.val = value;
            return;
        }
        current = current.next;
    }
```

```
    if (value % 2 == 0) {
```



```

    Node newNode = new Node(key, value);
    newNode.next = head;
    ht[index] = newNode;
}
else {
    Node newNode = new Node(key, value);
    current = head;

    while (current.next != null) {
        current = current.next;
    }
    current.next = newNode; // append
}
}

```

You are asked to implement part of a **Hashtable** class where the keys are integers and the values are strings. The class has an array of Node type named **ht**, which is considered the hashtable. You have to write the class method **insert**, which should insert a new key-value pair in the hashtable.

The class method **hash(key)** method computes the hash index based on the key and returns the index of the hashtable. This method is already written for you. You shall use this while inserting a key-value pair.

While inserting, if there's any collision, you should follow the separate chaining approach; however, you should make sure that the nodes are added in ascending order of the keys. If there's any duplicate key, replace the previous value of that corresponding key.

Make sure that the time complexity of your code is $O(n)$

The Node class (non-dummy-headed singly type) for the hashtable is already implemented with key (int), val (String), and next (Node) variables. You do not need to write it.

Initial Hashtable ht	After executing insert(8, "GOAT")
Index 0: (3, CAT) -> (4, EELS) Index 1: (5, LION) Index 2: (2, DOG) -> (9, CROW) Index 3: (27, WOLF) Index 4: (10, FISH) -> (11, OWL)	Assume 8 is mapped at index 2. Index 0: (3, CAT) -> (4, EELS) Index 1: (5, LION) Index 2: (2, DOG) -> (8, GOAT) -> (9, CROW) Index 3: (27, WOLF) Index 4: (10, FISH) -> (11, OWL)

Use the variable named **"ht"** to access the hashtable inside the method insert.

# Python notation	// Java notation
<pre>def hash(key): # use this function # already written def insert(key, value): # Write your code here</pre>	<pre>int hash(int key){ // use this function // already written } void insert(int key, String value){ // Write your code here }</pre>

```
void insert(int key, String value) {
```

```
int index = hash(key);
Node head = ht[index];
```

```
if (head == null) {
    ht[index] = new Node(key, value);
    return;
}
```

// Case 2: check if key should become new head OR update existing head

```
if (key == head.key) {           // same key → replace
    head.val = value;
    return;
}
```

```
if (key < head.key) {           // insert before head
    Node newNode = new Node(key, value);
    newNode.next = head;
    ht[index] = newNode;
    return;
}
```

// Case 3: insert somewhere after head

```
Node current = head;
while (current.next != null && current.next.key < key) {
    current = current.next;
}
```

// If duplicate key exists in the middle/tail → update value

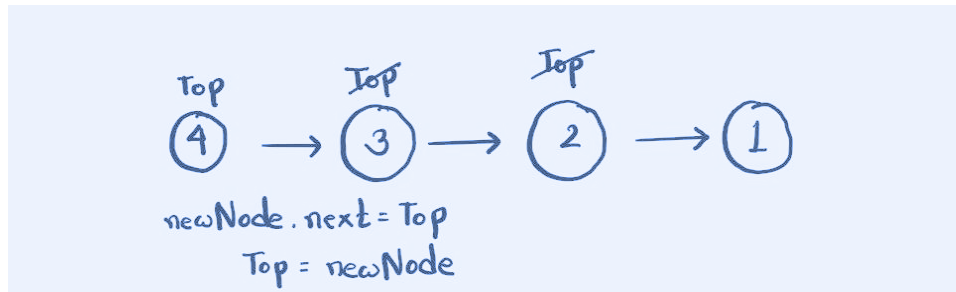
```
if (current.next != null && current.next.key == key) {
    current.next.val = value;
    return;
}
```

// Insert new node between current and current.next

```
Node newNode = new Node(key, value);
newNode.next = current.next;
current.next = newNode;
}
```

Stack Basics

Push



//Using linked list

```
public void push(int x) {  
    Node newNode = new Node(x);  
    if(top==null){  
        top=newNode;  
    }  
  
    else{  
        newNode.next = top;  
        top = newNode;  
    }  
}
```

OR

//Using Array

```
public void push(int x) {  
    if (top == capacity - 1) {  
        System.out.println("Stack Overflow");  
        return;  
    }  
    arr[++top] = x;           //Stack can't be circular  
}
```

Pop

//Using Linked list

```
public int pop() {
```

```

if(top==null){
    System.out.println("Stack underflow");
}
else{
    int val = top.elem;
    top = top.next;
    return val;
}
}

```

OR

```

//Using array
public int pop() {
    if (top == -1) {
        System.out.println("Stack Underflow");
        return -1;
    }
    return arr[top--];
}

```

Peek

```

//Using Linked list
public int peek() {
    if(top==null){
        System.out.println("Stack underflow");
    }
    else{
        return top.data;
    }
}

```

OR

```

//Using Array

```

```
public int peek() {
    if (top == -1) {
        System.out.println("Stack Underflow");
        return -1;
    }
    return arr[top];
}
```

Count elements in a stack

```
public static int countElements(Stack stack) {
    Stack temp = new Stack();
    int count = 0;

    while (!stack.isEmpty()) {
        Object topItem = stack.pop();
        temp.push(topItem);
        count = count + 1;
    }

    // This restores the original stack to its initial state
    while (!temp.isEmpty()) {
        Object itemFromTemp = temp.pop();
        stack.push(itemFromTemp);
    }

    return count;
}
```

Reversing

```
public static Stack reverseStack(Stack stack) {
    Stack temp = new Stack();

    while (!stack.isEmpty()) {
```

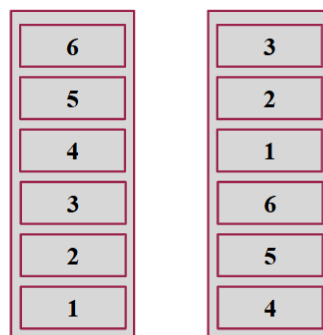
```

    temp.push(stack.pop());
}

return temp;
}

```

Mid Reversal of Stack



36

```

public static Stack reverseMid(Stack stack){
    Stack temp = new Stack();
    int count = 0;

    // Count elements
    while(!stack.isEmpty()){
        temp.push(stack.pop());
        count++;
    }

    Stack temp2 = new Stack();
    int n = 0;

    // Reverse the lower half
    while(n < count/2){
        temp2.push(temp.pop());
        n++;
    }

    while(!temp.isEmpty()){
        stack.push(temp.pop());
    }
}

```

```

while(!temp2.isEmpty()){
    stack.push(temp2.pop());
}

return stack;
}

```

Stack Application - Parenthesis Matching

(a (b + c) + d)

[(a b) (c d)]

([a { x y } b])

Correct

(a (b + c) + d

[(a b] (c d))

([a { x y) b])

Incorrect

```

public static boolean isBalanced(String expr) {
    Stack temp = new Stack();

    for (int i = 0; i < expr.length(); i++) {
        char ch = expr.charAt(i);

        if (ch == '(' || ch == '{' || ch == '[') {
            temp.push(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (temp.isEmpty()) {
                return false;
            }

            char top = (char) temp.pop();
            if ((top == '(' && ch != ')') ||
                (top == '{' && ch != '}') ||
                (top == '[' && ch != ']')) {

```



```

        return false;
    }
}

```

```

if(stack.isEmpty()){
    return true;
}
else{
    return false;
}
}

```

Infix to Post fix

Infix expression	Postfix expression
$(A + B) * (C - D)$	$AB + CD - *$
$(A + B) / (C - D) - (E * F)$	$AB + CD - / EF * -$

```

private static int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    if (op == '^') return 3;
    return 0; // for non-operators
}

```

```

public static String convert(String infix) {
    String postfix = "";

```

```

Stack stack = new Stack();

for (int i = 0; i < infix.length(); i++) {
    char ch = infix.charAt(i);

    //Operand
    if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z') || (ch >= '0' && ch <= '9')) {
        postfix += ch;
    }

    //Both Bracket
    else if ( ch == '(' ) {
        stack.push(ch);
    }

    else if ( ch == ')' ) {
        while (!stack.isEmpty() && (char)stack.peek() != '(') {
            postfix += (char)stack.pop();
        }
        stack.pop();        //remove first bracket
    }

    //Operator
    else {
        while (!stack.isEmpty() && precedence(ch) <= precedence( (char)stack.peek() ) ) {
            postfix += (char)stack.pop();
        }
        stack.push(ch);
    }
}

while (!stack.isEmpty()) {    //Outside of for loop
    postfix += (char)stack.pop();
}

return postfix;
}

```

Postfix Expression	Result
4 5 7 2 + - *	-16
3 4 + 2 * 7 /	2
5 7 + 6 2 - *	48
4 2 3 5 1 - + * +	18
4 2 + 3 5 1 - * +	18

```
public static int evaluate(String postfix) {
    Stack stack = new Stack();

    for (int i = 0; i < postfix.length(); i++) {
        char ch = postfix.charAt(i);

        // Operand
        if ((ch >= '0' && ch <= '9') || (ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {
            int num = Character.getNumericValue(ch);
            stack.push(num);
        }

        // Operator
        else {
            int b = (int) stack.pop();
            int a = (int) stack.pop();
            int res = 0;

            if (ch == '+') res = a + b;
            else if (ch == '-') res = a - b;
            else if (ch == '*') res = a * b;
            else if (ch == '/') res = a / b;
            else if (ch == '^') {

                stack.push(res);
            }
        }
    }
    return (int) stack.pop();
}
```

}

S Lab Task

2. Stack: Tower of Blocks:

A kid is trying to make a tower using blocks with numbers or characters written on them. He puts a block on the top of another to make the tower. As for removing the blocks, he removes the topmost block if needed. He is afraid that removing other blocks from the middle in one go will result in the collapse of his tower.



Now his friend wants your nth block from the top of your tower. He is willing to give him the nth block from the top, preserving the relative position of others.

Following his tower build process, solve this scenario using **Stack** class. You can only use the stack methods.

Hints: You can create a temporary object of Stack class.

In the following examples, consider the rightmost element to be the topmost element of the stack. (**Rightmost item is the top**)

Sample Tower	Sample Output	Explanation
Stack: 4 19 23 17 5 n: 2	Stack: 4 19 23 5	In this tower, the 2nd block from the top is the block with 17. After removing the block the stack looks like 4 19 23 5
Stack: 73 85 15 41 n: 3	Stack: 73 15 41	In this tower, the 3rd block from the top is the block with 85. After removing the block the stack looks like 73 15 41

```
public static void removeBlock(Stack stack, int n) {
```

```
    // Create a temporary stack to hold items
```

```
    Stack temp = new Stack();
```

```
    // Move the first (n-1) blocks to temp stack
```

```
    int count = 1;
```

```
    while (count < n) {
```

```
        Object topItem = stack.pop();
```

```
        temp.push(topItem);
```

```
        count = count + 1;
```

```
    }
```

```
    //Pop the latest
```

```
    stack.pop();
```

```
    //Re add
```

```
    while (!temp.isEmpty()) {
```

```
        Object itemFromTemp = temp.pop();
```

```
        stack.push(itemFromTemp);
```

```
    }
```

}

S Lab Assignment

2. Stack & Queue: Validate and Evaluate a Mathematical Expression

Complete the function named **evalMathExpression** which will take a mathematical expression as a string and output the evaluated value.

The expression may contain:

- Integer numbers
- Arithmetic operators: +, -, *, /, ^
- Parentheses/braces: (), [], {}

Your task is to:

- Check whether the parentheses are balanced and properly nested.
- If the parentheses are balanced:
 - Convert the infix expression to its postfix form.
 - Print the postfix expression.
 - Evaluate the postfix expression and print the final result.
- If the parentheses are invalid, output "Invalid Expression".

NB: You may use only one Stack and one Queue object to solve this problem. No other data structures are allowed.

Input	Output
"3 + 5 * (2 - 8) "	Postfix: 3 5 2 8 - * + Result: -27
"(2 + 3)) * ((4 - 1) "	Invalid Expression
"7 + (6 * 5^2 + 3) - (4 / 2) "	Postfix: 7 6 5 2 ^ * 3 + + 4 2 / - Result: 158
"(10 + 2) * 6 / 3"	Postfix: 10 2 + 6 * 3 / Result: 24
"[2 + 3) * (4 - (5 * 6))] "	Invalid Expression

```
private static int precedence(char a){
    if(a=='+'|| a=='-'){
        return 1;
    }

    if(a=='*'|| a=='/'){
        return 2;
    }

    if(a=='^'){
        return 3;
    }

    return -1;
}
```

```

public static void evalMathExpression(String expression) {
    Stack temp= new Stack();
    boolean valid=true;

    //Bracket check
    for(int i=0; i<expression.length(); i++){
        char a=expression.charAt(i);
        if(a=='(' || a=='{' || a=='['){
            temp.push(a);
        }

        else if(a==')' || a=='}' || a==']'){
            if(temp.isEmpty()){
                valid=false;
                break;
            }

            char top=(char)temp.pop();
            if(a==')' && top!='(' || a=='}' && top!='{' || a==']' && top!='['){
                valid=false;
                break;
            }
        }
    }

    if(valid==false){
        System.out.println("Invalid Expression ");
        return;
    }
}

```

```

else{

```

```

    //Infix to postfix
    LinkedListQueue postfix=new LinkedListQueue();
    for(int i=0; i<expression.length(); i++){

```

```

char a=expression.charAt(i);

if(a>='0' && a<='9'){
    postfix.enqueue(a);
}
else if(a=='('){
    temp.push(a);
}

else if(a=='){
    while(temp.isEmpty()==false && (char)temp.peek()!='(' ){
        char p=(char)temp.pop();
        postfix.enqueue(p);
    }
    temp.pop();    //Start bracket remove
}

else{
    while(!temp.isEmpty() && precedence(a)<=precedence((char)temp.peek())){
        char p=(char)temp.pop();
        postfix.enqueue(p);
    }
    temp.push(a);    //Beshi or equal precedence ber kore notun ta boshano
}
}

while(temp.isEmpty()==false){
    char p=(char)temp.pop();    //Extra baki thakle
    postfix.enqueue(p);
}

//Print
System.out.print("Postfix: ");
LinkedListQueue temp_postfix= new LinkedListQueue(); //underflow hobe without temp

while(postfix.isEmpty()==false){
    Object value= postfix.dequeue();
    temp_postfix.enqueue(value);
    System.out.print(value+ " ");
}

System.out.println();

```

```

//Infix sum
while(temp_postfix.isEmpty()==false){
    char a=(char)temp_postfix.dequeue();

    if(a>='0' && a<='9'){
        int num= Character.getNumericValue(a);
        temp.push(num);
    }

    else{
        int y=(int)temp.pop();
        int x=(int) temp.pop();
        int result=0;

        if(a=='+'){
            result=x+y;
        }
        else if(a=='-'){
            result=x-y;
        }
        else if(a=='*'){
            result=x*y;
        }
        else if(a=='/'){
            result=x/y;
        }
        else if(a=='^'){
            result=(int)Math.pow(x,y);
        }
        temp.push(result);
    }
}
System.out.println((int)temp.pop());

}
}

```

2. Stack Reverse:

Consider that a Stack class has been implemented containing the push(element), pop(), peek() and isEmpty() functions.

Complete the function **conditional_reverse()** which will take an object of Stack class as an input that contains some integer values. **The function returns a new stack** which will contain the values in reverse order from the given stack **except the consecutive one's**. You cannot use any other data structure except Stack.

Note: The Stack class implements a singly linked list-based Stack hence overflow is not possible. The pop() and peek() functions return None in case of the underflow.

In the following example, consider the rightmost element to be the topmost element of the stack.

Sample Input Stack (Rightmost is the top)	Output Reversed Stack (Rightmost is the top)	Explanation
Stack: 10, 10, 20, 20, 30, 10, 50 Top = 50	New Stack: 50, 10, 30, 20, 10 Top = 10	Consecutive 20 and 10 are not present in the output reversed stack

```
public static Stack conditionalReverse(Stack stack) {  
    Stack temp=new Stack();  
    while(!stack.isEmpty()){  
        int a= stack.pop();  
        if(!stack.isEmpty() && a!=stack.peek()){  
            temp.push(a);  
        }  
        else{  
            temp.push(a); //Add last element  
        }  
    }  
    return temp;  
}
```

S Rfts Practice

You are given the start and end time of some tasks. The tasks are sorted according to their starting time. You can merge two tasks **A** and **B** if they are overlapping. That is task **B** starts before the finishing time of task **A**. In that case, it becomes a single task having the ending time of task **A** or task **B** (depending upon which task finishes later). This task can be merged further with other tasks if the above condition satisfies (overlapping). You want to merge as many tasks as possible and minimize the total number of non-overlapping tasks.

You are given a 2D array named *tasks* storing the start and end time of the task. The array is sorted according to the starting time of the tasks. Each row of the array stores two integer values. The first value indicates the start time and the second value indicates the end time of a task. More specifically, the start and end time of the *i*th task is *tasks[i][0]* and *tasks[i][1]* respectively.

Write a function named *print_total_task(tasks)* that takes the tasks array as input and prints the start time and end time for each non-overlapping task according to the given format. Note that the tasks are printed as decreasing order of start time.

Consider that the Stack class is already available with the following methods/functions implemented: *push(element)*, *pop()*, *peek()*, and *isEmpty()*. Stack Underflow and Stack Overflow exceptions return None (Python) / null(Java). There is no need to write the Stack class.

You are allowed to create as many *stacks* as you want. You cannot create any new array. To create a Stack object use the following:

stack = Stack() #(Python) **Stack stack = new Stack()** //(Java)

Example 1:	Explanation:
Input: <i>tasks</i> = [[1, 5], [2, 3], [4, 6], [7, 10], [9, 11], [12, 15]]	Tasks (1,5) and (2,3) are overlapping, so we can merge them creating the new task (1,5) Then task (1,5) and (4,6) are also overlapping, merging them creates the new task (1,6) We can not merge tasks (1,6) and (7,10) as they are non-overlapping However tasks (7,10) and (9,11) can be merged creating new task (7, 11) Finally tasks (7,11) and (12,15) are non-overlapping
Output: 12, 15 7, 11 1, 6	So we have 3 non-overlapping tasks (1,6), (7,11) and (12,15) Tasks are printed in descending order of start time.

```
public static void printTotalTask(int[][] tasks) {
    Stack temp = new Stack();
    temp.push(tasks[0]);

    for (int i = 1; i < tasks.length; i++) {
        int[] top = (int[]) temp.peek();
        int[] current = tasks[i];

        if (current[0] <= top[1]) {
            if (current[1] > top[1]) {
                top[1] = current[1];
            }
        } else {
            temp.push(current);
        }
    }
}
```

```
// Print in descending order of start time
Stack reverse = new Stack();
while (!stack.isEmpty()) {
    reverse.push(stack.pop());
}

while (!reverse.isEmpty()) {
    int[] t = (int[]) reverse.pop();
    System.out.println(t[0] + ", " + t[1]);
}
}
```

Practise Problem

You are given a stack containing integers in arbitrary order. Your task is to write a function that segregates odd and even numbers while maintaining their relative order and finally returns the stack.

- Odd numbers should appear at the top (in the same order as in the original stack).
- Even numbers should appear at the bottom (in the same order as in the original stack).
- You can only use stack operations: push(value), pop(), peek(), and isEmpty().
- You may use multiple auxiliary stacks but cannot use other data structures like arrays or lists.
- For Overflow and Underflow exceptions, assume the methods return None (Python) / null (Java)

Stack Creation Example

Python	Java
stack = Stack()	Stack stack = new Stack()

Input Stack	Modified Stack	Explanation
12 7 5 8 11 14 3 <-- Top	7 5 11 3 12 8 14 <-- Top	The odd numbers (7,5,11, 3) are grouped together and appear at the top of the output stack, in the same order as the original stack. The even numbers appear next, maintaining their original order.
1 2 3 4 5 6 7 8 <-- Top	1 3 5 7 2 4 6 8 <-- Top	Same reasoning as before

```
public static Stack segregateOddEven(Stack input) {
    Stack even = new Stack();
    Stack odd = new Stack();

    while (!input.isEmpty()) {
        int x = (int) input.pop();
```



```
    if (x % 2 == 0) {
        even.push(x);
    } else {
        odd.push(x);
    }
}

while (!even.isEmpty()) {
    input.push(even.pop());
}

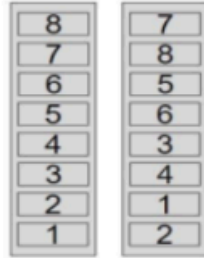
while (!odd.isEmpty()) {
    input.push(odd.pop());
}

return input;
}
```

You are given a stack of some integer elements, st. Your task is to swap the adjacent elements of the stack and return the modified stack. If the number of elements in the stack is odd, keep the topmost element as it is and swap the other adjacent elements. See the input output for better understanding. Now write a function **Do_Adjacent_Swap(st)** that completes the given task and returns the modified stack.

The st is an object of the "Stack" class which is implemented by a singly linear linked list and consider that the following methods of "Stack" class are already implemented for you: **push(element)**, **pop()**, **peek()**, **isEmpty()**.

You cannot use any other data structures other than stack. The "Top" pointer of Stack class is NOT accessible to you. You can use just the above four methods of Stack class. You can create as many stacks as you need for this task.

Sample Input	Sample Output	Explanation
Original Stack: 8(Top) → 7 → 6 → 5 → 4 → 3 → 2 → 1	Modified Stack: 7(Top) → 8 → 5 → 6 → 3 → 4 → 1 → 2	

```

public static void Do_Adjacent_Swap(Stack st) {
    Stack temp = new Stack();

    while (!st.isEmpty()) {           //Extra temp jate correct sequence e boshate pari
        temp.push(st.pop());
    }

    while (!temp.isEmpty()) {
        int first = (int) temp.pop();

        if (temp.isEmpty()) {
            st.push(first);
        }
        else {
            int second = (int) temp.pop();
            st.push(second);
            st.push(first);
        }
    }
}

```

Ascending order in Stack

```
public static void sortStack(Stack st) {
    Stack temp = new Stack(); // helper stack

    while (!st.isEmpty()) {
        int x = (int) st.pop();

        while (!temp.isEmpty() && (int) temp.peek() > x) {
            st.push(temp.pop());
        }

        temp.push(x);
    }

    while (!temp.isEmpty()) {
        st.push(temp.pop());
    }
}
```

Input Stack	Modified Stack	Explanation													
stack = Top →<table><tr><td>12</td></tr><tr><td>-3</td></tr><tr><td>8</td></tr><tr><td>15</td></tr><tr><td>-7</td></tr><tr><td>-20</td></tr><tr><td>-6</td></tr><tr><td>-2</td></tr><tr><td>9</td></tr><tr><td>14</td></tr></table> window = 4	12	-3	8	15	-7	-20	-6	-2	9	14	stack = Top →<table><tr><td>35</td></tr><tr><td>0</td></tr><tr><td>23</td></tr></table>	35	0	23	Since the window size is given as 4, each window will have 4 values (except possibly the last one, if there are fewer remaining elements). Window 1: has [12, -3, 8, 15] Sum of deposits: 12 + 8 + 15 = 35 Window 2: has [-7, -20, -6, -2] Sum of deposits: 0 (Sum is 0 since no deposits in this window) Window 3: has [9, 14] (remaining 2 elements) Sum of deposits: 9 + 14 = 23 Modified stack means you must return the stack you received via the function parameter.
12															
-3															
8															
15															
-7															
-20															
-6															
-2															
9															
14															
35															
0															
23															

```
public static Stack windowDeposit(Stack st, int window) {
```

```
    Stack temp = new Stack();
```

```
    while (!st.isEmpty()) {
```

```
        int count = 0;
```

```
        int sum = 0;
```

```
        while (count < window && !st.isEmpty()) {
```

```
            int val = (int) st.pop();
```

```
            if (val > 0) {
```

```
                sum += val;
```

```
            }
```

```
            count++;
```

```
        }
```

```
        temp.push(sum);
```

```
    }
```

```
    while (!temp.isEmpty()) {
```

```
        st.push(temp.pop());
```

```
    }
```

```
    return st;
```

```
}
```

Advanced

Rat in a Maze

Input: mat[][] = [[1, 0, 0, 0], [1, 1, 0, 1], [1, 1, 0, 0], [0, 1, 1, 1]]

Source			
			Dest.

```
public static boolean pathExists(int[][] maze, int fx, int fy) {
```

```
    int n = maze.length;
    int m = maze[0].length;
```

```
    boolean[][] visited = new boolean[n][m];
```

```
    Stack sx = new Stack(); // index of x-coordinates
    Stack sy = new Stack(); // index of y-coordinates
```

```
    if (maze[0][0] == 0) return false;
```

```
    sx.push(0); // store index
    sy.push(0);
    visited[0][0] = true;
```

```
    while (!sx.isEmpty()) {
```

```
        int x = (int) sx.pop();
        int y = (int) sy.pop();
```

```
        // reached food!
        if (x == fx && y == fy) return true;
```

```
        //UP
        if (x - 1 >= 0 && maze[x-1][y] == 1 && !visited[x-1][y]) {
            visited[x-1][y] = true;
            sx.push(x-1);
            sy.push(y);
        }
```

```

// DOWN
if (x + 1 < n && maze[x+1][y] == 1 && !visited[x+1][y]) {
    visited[x+1][y] = true;
    sx.push(x+1);
    sy.push(y);
}

// LEFT
if (y - 1 >= 0 && maze[x][y-1] == 1 && !visited[x][y-1]) {
    visited[x][y-1] = true;
    sx.push(x);
    sy.push(y-1);
}

// RIGHT
if (y + 1 < m && maze[x][y+1] == 1 && !visited[x][y+1]) {
    visited[x][y+1] = true;
    sx.push(x);
    sy.push(y+1);
}
}

return false;
}

```

Queue Basics

Array Based

```
public void enqueue(int x) {
    if (size == capacity) {
        System.out.println("Queue Overflow");
        return;
    }
    rear = (rear + 1) % capacity;
    arr[rear] = x;
    size++;
}

public int dequeue() {
    if (size == 0) {
        System.out.println("Queue Underflow");
        return -1;
    }
    int val = arr[front];
    front = (front + 1) % capacity;
    size--;
    return val;
}

public int peek() {
    if (size == 0) return -1;
    return arr[front];
}
```

LinkedList based

```
public void enqueue(int x) {
    Node n = new Node(x);
    if (rear == null) {
```

```
        front = rear = n;  
        return;  
    }  
    rear.next = n;  
    rear = n;  
}
```

```
public int dequeue() {  
    if (front == null) {           //Ektao nai  
        System.out.println("Underflow");  
        return -1;  
    }  
    int val = front.data;  
    front = front.next;  
    if (front == null) rear = null;    //Just ekta chilo  
    return val;  
}
```

```
public int peek() {  
    if (front == null) return -1;  
    return front.data;  
}
```

You need to perform the following operations on an array-based queue where the length of the array is 4 and the starting index of front and back is 3. [Blank means empty space].

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue.

Index →	0	1	2	3	Front index	Back Index
Initial					3	3
enq (a)				a	3	0
enq (b)	b			a	3	1
deq	b				0	1
peek	b				0	1
enq (c)	b	c			0	2
peek	b	c			0	2
deq		c			1	2

Q Lab Task

3. Queue: Customer Service Call Center:

You are developing a **Customer Service Call Center System** where customers call for assistance and are placed in a queue based on a **first-come-first-served** basis. However, there are two types of customers:

- i. **Regular Customers**
- ii. **VIP Customers** (who should be served before regular customers but in the order they arrive among VIPs).

Your task is to implement a **call handling system** using a queue that follows these rules:

- i. **VIP customers get priority** over regular customers.
- ii. Among VIP customers, they are served in the order they are called (FIFO).
- iii. Among regular customers, they are also served in the order they are called (FIFO).
- iv. When a call is handled, remove the customer from the queue and process them.

Implement the **CallQueue** class using a queue data structure that supports the following methods:

- **enqueue_call(customer_id, is_vip) →**
 - Adds a customer to the queue. If `is_vip = True`, they should be placed in the VIP queue; otherwise, in the regular queue.
- **dequeue_call() →**
 - Processes the next call. It should first check for VIP customers, and if none exist, then process regular customers.
- **display_queue() →**
 - Prints the current order of the queue (VIPs first, followed by regular customers).

[NOTE: A `LinkedListQueue` class is already given in the driver code, you just need to utilize it in your `CallQueue` class to complete the tasks]

```
public void enqueueCall(int customerId, boolean isVip) {
    if (isVip==true) {
        vipQueue.enqueue(customerId);
    }
    else {
        regularQueue.enqueue(customerId);
    }
}
```

```
public void dequeueCall() {
    if (vipQueue.isEmpty()==false) {
```

```
        vipQueue.dequeue();
    }
    else if (regularQueue.isEmpty()==false) {
        regularQueue.dequeue();
    }
    else {
        System.out.println("No calls in the queue.");
    }
}
```

```
public void displayQueue() {
    vipQueue.displayQueue();
    regularQueue.displayQueue();
}
```

Lab Q Assignment

3. Queue: Remove Consecutive Duplicates

NOTE: YOU MUST USE QUEUE TO SOLVE THIS TASK

Write a method/function called **removeConsecutiveDups()** that takes only one String in its parameter. You are given a String which may or may not contain consecutive duplicate values. Finally the **removeConsecutiveDups()** method returns a new String containing no consecutive duplicate characters. If no duplicates are found then it simply returns the same string.

Sample Given String	Sample Returned String
aabbbccccdd	abcd
aaabbaa	aba
abcabcabc	abcabcabc
aaaaa	a

```
public static String removeConsecDups(String word){
    LinkedListQueue q=new LinkedListQueue();
    String st="";

    char prev= word.charAt(0);
    q.enqueue(prev);

    for(int i=1; i<word.length(); i++){
        char current=word.charAt(i);
        if((current==prev)==false){
            q.enqueue(current);
            prev=current;    //Only enqueue hole shift hobe
        }
    }

    while(q.isEmpty()==false){
        st+=q.dequeue();
    }

    return st;
}
```


Rfts Practice

You are given a **Stack** of people. They should be paired off as man with woman, woman with man. If a male and a female are consecutive in the stack, they get paired immediately. If there are consecutive females found without any subsequent males, then the spare females should be saved in a **Queue** for future pairing. If there are consecutive males found without any subsequent females, we try to pair a male with the first spare female [if available] from the **Queue**. If no spare female is available, then the male is discarded. The same logic applies to consecutive females without any subsequent males.

The stack class and the Queue class are implemented for you. **The public instance methods in the Stack class are push(), pop(), peek(), and isEmpty(). The public instance methods in the Queue class are enq(), deq(), peek(), and isEmpty().** In both classes, isEmpty() returns a boolean value, peek() and pop() returns None for underflow. All other members of both classes are private. You can only use the public instance methods. No need to consider the overflow exception for both Stack and Queue classes.

Your task is to complete the given method **dance_pair()** that takes a stack and prints every male-female pair. **Given:**

<pre>def id_gender_extractor(s): if s != None: return s[0],s[2:] #gender, id else: return None, None</pre>	Driver Code: <pre>st = Stack() st.push('F_18') st.push('F_9') st.push('M_7') st.push('M_1') st.push('M_19') st.push('M_3') st.push('F_5') st.push('F_4') st.push('F_20') st.push('M_10')</pre> dance_pair(st)
--	--

Every male and female are written in this way - M_id or F_id. A method called **id_gender_extractor(s)** is implemented for you that returns the id and gender from a string s.

You do not need to write the methods/classes that are implemented for you. You cannot use any data structure other than stack and queue.

Sample Input	Output	Explanation
st = Top → M_10 F_20 F_4 F_5 M_3 M_19 M_1 M_7 F_9 F_18	10 and 20 are paired together 5 and 3 are paired together 4 and 19 are paired together 7 and 9 are paired together	M_10 and F_20, F_5 and M_3, M_7 and F_9 are sequential in the stack. They are paired up. From F_4, there are consecutive females. So, F_4 is stored in the queue for future pairing. From M_19, there are consecutive males. M_19 is paired with spare F_4. M_1 is discarded since there is no spare female with whom M_1 can be paired.

Practise Problem

24

```
static String getGender(String s) {
    return s.charAt(0) + ""; // "M" or "F"
}
```

```
static String getId(String s) {
    return s.substring(2); // after "_ "
}
```

```
public static void dance_pair(Stack st) {
    Queue spareFemales = new Queue();

    while (!st.isEmpty()) {
        String current = st.pop();

        // 2nd pop
        if (!st.isEmpty()) {
            String next = st.pop();

            //Consecutive
            if (!getGender(current).equals(getGender(next))) {
```

```
        System.out.println(getId(current) + " paired with " + getId(next));
    }
}

// If male
if (getGender(current).equals("M")) {
    if (!spareFemales.isEmpty()) {
        System.out.println(id + " paired with " + spareFemales.deq());
    }

    // If female
else {
    spareFemales.enq(current);
    spareFemales.enq(next);
}
}
}
```